Robust License Plate Detection and Recognition Using YOLO-Based Oriented Object Detection

Bao Le Duc Gia
Ho Chi Minh City, Vietnam

Tri Huynh Quang
Ho Chi Minh City, Vietnam

Long Vu Nguyen Minh
Ho Chi Minh City, Vietnam

Hung Nguyen Viet
Ho Chi Minh City, Vietnam

***Abstract*—With the rapid expansion of intelligent transportation systems, the demand for robust License Plate Recognition (LPR) in unconstrained environments has intensified. However, traditional deep learning approaches often struggle with the extreme camera angles and diverse plate formats typical of Vietnamese traffic. This paper proposes a high-precision LPR framework integrating state-of-the-art YOLO-based Oriented Object Detection (OBB) models, including YOLOv8n, YOLOv11n, and YOLOv26n. The implementation of OBB enables precise plate localization and rectification from challenging perspectives, which are subsequently processed via the PaddleOCR engine. To further enhance reliability, we introduce a Syntax-Based Correction (SBC) module tailored to Vietnamese alphanumeric plate standards. Experimental results demonstrate that the system achieves a peak mean Average Precision (mAP_{50}) of 99.23% for detection and an end-to-end plate recognition accuracy of 70.98%. The proposed framework significantly outperforms baseline models by resolving common character ambiguities, providing a viable solution for real-time campus security and urban monitoring.**

***Index Terms*—License Plate Recognition, Object Detection, YOLO, Oriented Bounding Box, PaddleOCR**

I. Introduction

The convergence of artificial intelligence and computer vision has revolutionized urban security, particularly through the deployment of Intelligent Transportation Systems (ITS). A cornerstone of these technologies is the Automated License Plate Recognition (ALPR) system, which facilitates automated vehicle tracking, parking management, and law enforcement. However, the efficacy of ALPR is frequently compromised by environmental variables such as low illumination, motion blur, and perspective distortion. In the specific context of Vietnam, these challenges are exacerbated by exceptionally high motorcycle density and a wide variety of license plate formats captured from non-orthogonal CCTV angles.

Traditional horizontal bounding box (H-Box) detectors often fail in these scenarios, as they include excessive background noise when plates are tilted, leading to significant errors in the subsequent Optical Character Recognition (OCR) stage. To address these deficiencies, this paper proposes a robust LPR framework optimized for Vietnamese conditions using Oriented Bounding Boxes (OBB). By predicting the four-corner coordinates of the plate, our system ensures a tight spatial fit regardless of the camera's viewing angle.

This study evaluates three cutting-edge nano-architectures—YOLOv8n-OBB, YOLOv11n-OBB, and YOLOv26n-OBB—to identify the optimal balance between inference speed and detection accuracy. Following localization, we employ the PaddleOCR engine for character extraction. A key contribution of this work is the development of a Syntax-Based Correction (SBC) module. This post-processing layer utilizes domain-specific knowledge of Vietnamese plate structures to rectify common OCR misinterpretations (e.g., confusing '0' and 'D'). Our integrated approach, validated on a multi-source dataset including self-collected campus footage, demonstrates high robustness and real-time capability, offering a significant performance boost over standard detection pipelines.

II. System Architecture

The proposed system architecture is designed as a sequential inference pipeline that transforms raw unstructured traffic video into structured vehicle data. The pipeline consists of five primary stages:

Fig. 1. Overall pipeline of the proposed license plate detection and recognition system.

- **Data Acquisition and Slicing**: The system accepts high-definition traffic video streams as input. To enable real-time processing, the video is sliced into discrete image frames at a predefined sampling rate (e.g., every 15 frames) to reduce temporal redundancy while maintaining environmental context.
- **Object Detection**: Each frame is processed by the YOLO model to perform real-time plate localization. Unlike standard detection, this stage generates Oriented Bounding Boxes (OBB) to precisely encapsulate license plates, even when they are tilted or viewed from extreme angles.
- **Dynamic Cropping**: Once localized, the region of interest (ROI) containing the license plate is dynamically cropped from the original frame. This step isolates the plate from background noise, such as vehicle bodies

or road textures, to prepare the image for character extraction.

- **Quality Filtering**: To ensure high recognition accuracy, a filtering layer is applied to the cropped images. This stage identifies and discards poor-quality crops—such as those with excessive motion blur, severe occlusion, or low resolution—that would otherwise lead to OCR errors.
- **Inference and Output**: The filtered plate images are fed into the PaddleOCR engine for character extraction. The final output overlays the recognized license plate number onto the original video frame and logs the data into a structured format (e.g., CSV) for querying via the integrated chatbot interface.

III. Dataset and Data Collection

A. Data Acquisition and Dataset Characteristics

To ensure generalizability, a multi-source dataset was curated:

- **Public Repositories**: 7,044 images were sourced from GitHub as a foundation.
- **Self-Collected Data**: 33 videos were captured at FPT University (FPTU) campus, including raw videos simulating CCTV angles and multi-angle smartphone images.
- **Pre-processing**: Videos were sliced into frames (every 15 frames) to minimize redundancy while preserving environmental diversity.
- **In total:** 2353 images were in self-collected data

B. Data Cleaning and Integrity

To ensure the high fidelity of the training dataset, a rigorous cleaning and synchronization process was established. This phase focuses on maintaining a strict one-to-one correspondence between images and their respective spatial annotations.

- **Intersection Filter**: Since the raw dataset was aggregated from multiple sources (GitHub and self-collected), an intersection filter was applied to identify the common set of filenames present in both the image and label directories. This mechanism ensures that only valid image-label pairs are retained, resulting in 9,397 verified filenames for the final training pool.
- **Auto-renaming and Standardization**: To prevent filename collisions and manage dataset versions, an automated Python-based renaming script was deployed. All files were standardized to a sequential format (e.g., `car.jpg`), ensuring data integrity during the merge from various Google Drive folders.
- **Label Format Synchronization**: The raw dataset contained mixed annotation formats, including Horizontal Boxes, Polygons, and standard OBB. We developed a conversion pipeline to normalize all inputs into a rotated 4-corner coordinate system. For Polygon annotations, the OpenCV *minAreaRect* function with a 1000x scaling trick was used to compute the Minimum Rotated Rectangle.
- **Out-of-Bounds Protection**: A safety clipping mechanism using the function $max(0.0, min(1.0, c))$ was applied to all coordinates. This prevents training errors caused by bounding boxes extending beyond image boundaries due to perspective transforms or manual annotation errors.

C. Data Preprocessing and Label Standardization

We utilized Contrast Limited Adaptive Histogram Equalization (CLAHE) to improve character visibility against the plate background. Mixed label formats were synchronized into a 4-corner coordinate system. For polygons, we applied the OpenCV *minAreaRect* algorithm to compute the Minimum Rotated Rectangle for OBB training.

D. Dataset Examples

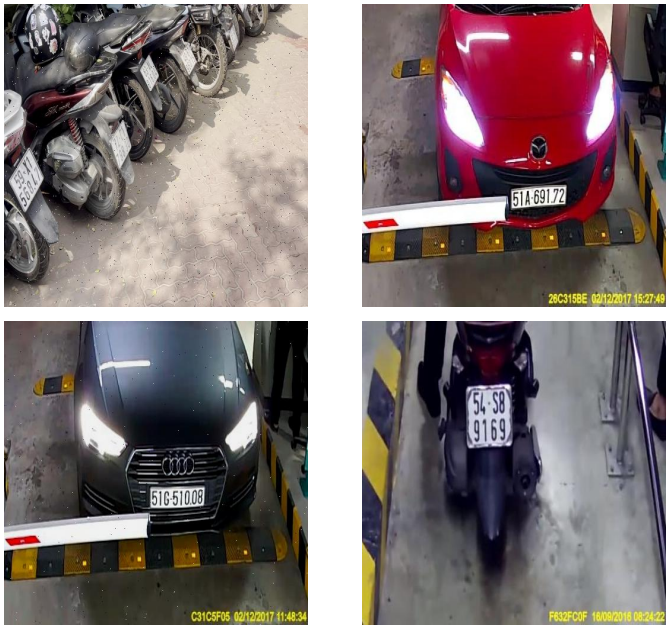

Fig. 2. Example images from the license plate dataset used for training.

IV. Feature Engineering and Data Augmentation

To enhance the robustness of the model and mitigate overfitting, we employed a comprehensive data augmentation strategy through the YOLO model. This approach allows for the generation of synthetic variations of the training data, simulating diverse real-world environmental conditions.

A. Preprocessing Steps

Before augmentation, standardized preprocessing was applied to the entire dataset to ensure input consistency:

- **Auto-Orient**: Applied to maintain the correct orientation of images across different camera sources.
- **Resizing**: All images were resized to a uniform dimension of 512×512 pixels using a stretch transformation to fit the model's input requirements.

B. Augmentation Parameters

The augmentation pipeline was designed to replicate the challenges of Vietnamese motorcycle parking environments, such as low-light conditions, motion blur, and sensor noise:

- **Grayscale**: Applied to 15% of the images to reduce dependency on color features, focusing the model on structural and textual patterns.
- **Brightness and Exposure**: Random adjustments between -15% and $+15\%$ for brightness, and -10% to $+10\%$ for exposure, simulating midday sun and evening shadows.
- **Blur and Noise**: To mimic low-quality CCTV footage, we added blur (up to 0.9px) and Gaussian noise (up to 1.5% of pixels).

Through these transformations, the original dataset was expanded. This significant increase in data volume provides the diverse samples necessary for the high-precision detection of tilted and distorted license plates.

V. Dataset Partition

The finalized dataset consists of a high-fidelity collection of 9397 valid image-label pairs before augmentation. The distribution is structured as follows:

- **Training Set**: Consists of 6577 base images.
- **Validation Set**: Used 1410 during training to monitor performance and prevent overfitting.
- **Testing Set**: Consists of 1410 images.

The split was performed using a fixed random_state = 42 to ensure reproducibility, focusing on an 70% Training, 15% Validation distribution and 15% Test.

As shown in Fig. 2, the dataset contains license plates captured under different lighting conditions, angles, and distances.

VI. License Plate Detection Model

The core of our detection system evaluates several YOLO-based oriented bounding box architectures, specifically configured to ensure high inference accuracy while maintaining hardware efficiency. To prevent hardware bottlenecks such as Out-of-Memory (OOM) errors and to ensure stable training, our modeling setup utilizes a dynamic resource allocation strategy.

A. Modeling Setup and Strategy

The training process is governed by several critical hyperparameters that define the model's learning behavior and hardware interaction:

```
results = model.train(
    data=yaml_path,
    epochs=50,        # Maximum number of
        training cycles
    imgsz=640,        # Input image resolution
    batch=16,         # Dynamic batch size to
        prevent OOM
    device=0,         # Utilize GPU acceleration
    workers=4,        # Multi-threading for data
         loading
    project='runs/train',
    name='plate_model_v1',
    patience=30       # Early stopping patience
)
```

Listing 1. Hyperparameter configuration for YOLO training.

- **Input Resolution**: All images are processed at a resolution of 640×640 pixels (`imgsz=640`), providing an optimal balance between the detection of small plate characters and computational load.
- **Epochs and Convergence**: We set a maximum of 50 training epochs. To optimize time and prevent overfitting, an **Early Stopping** mechanism is integrated with a *patience* value of 30 epochs, halting the process if no significant improvement is detected in the validation loss.
- **Adaptive Resource Allocation**:
  - **Batch Size**: We utilize a baseline batch size of 16 (`batch=16`). This parameter is adjusted dynamically based on the specific model variant (Nano, Small, or Medium) and the available VRAM to prevent hardware overflow.
  - **Multi-threading**: To accelerate the data loading process, we employ 4 CPU workers (`workers=4`). The number of workers is tuned according to the CPU's multi-core capability to ensure a steady data stream to the GPU.
- **Hardware Optimization**: Training is executed using GPU acceleration (`device=0`) to leverage CUDA cores for parallel processing.

VII. Experimental Results and Comparative Analysis

The performance of the three selected OBB (Oriented Bounding Box) models was evaluated using standard computer vision metrics. The quantitative results, demonstrating high precision and recall across all architectures, are summarized in Table I.

TABLE I
PERFORMANCE COMPARISON OF YOLO-BASED OBB DETECTION MODELS.

Model	Precision	Recall	mAP_{50}	mAP_{50-95}
YOLOv8n-OBB	0.9852	0.9847	0.9800	0.9494
YOLO11n-OBB	0.9841	0.9877	0.9923	0.9493
YOLO26n-OBB	0.9778	0.9728	0.9860	0.9419

A. Discussion of Findings

A comparative analysis of the experimental results reveals several key insights regarding the effectiveness of the evaluated nano-detection models:

- **Peak Precision vs. Recall: YOLOv8n-OBB** achieves the highest precision (0.9852), while **YOLO11n-OBB** leads in recall (0.9877) and mAP_{50} (0.9923). This indicates that YOLO11n is slightly more robust at capturing all potential plates in the frame, whereas YOLOv8n is marginally more accurate in its positive predictions.

- **Efficiency of YOLO26n:** Despite the slightly lower metrics compared to the v8 and v11 counterparts, **YOLO26n-OBB** maintains a competitive mAP_{50} of 0.9860. Given its architectural optimizations, it remains a viable candidate for edge deployment where computational resources are limited.
- **Localization Consistency:** The gap between mAP_{50} and mAP_{50-95} is consistent across all models (approx. 0.03–0.04). This suggests that the OBB implementation provides stable localization regardless of the model version, which is critical for the subsequent OCR alignment.

B. Recognition Results and Syntax-Based Correction

The recognition phase highlights the importance of domain-specific constraints. In the Vietnamese license plate system, motorcycles and automobiles follow a strict alphanumeric format (e.g., the 3rd character in a standard motorcycle plate must be a letter). We implemented a **Syntax-Based Correction (SBC)** module to post-process the OCR output.

TABLE II
IMPACT OF SYNTAX-BASED CORRECTION ON RECOGNITION ACCURACY.

Configuration	YOLOv8n	YOLO11n	YOLO26n	Metric
Raw Output	66.25%	65.54%	66.74%	Plate Acc.
	84.62%	84.62%	84.74%	Char Acc.
With SBC	70.12%	69.60%	70.98%	Plate Acc.
	85.71%	85.53%	85.98%	Char Acc.
Improvement (Δ)	**+3.87%**	**+4.06%**	**+4.24%**	Plate Acc.
	+1.09%	+0.91%	+1.24%	Char Acc.

- **Syntactic Correction Logic:** The SBC module identifies common OCR misinterpretations (e.g., mistaking '0' for 'D' or '1' for 'I') by validating the character type against the expected Vietnamese plate position. If a digit appears where a letter is mandatory, the module maps the character to its most likely alphanumeric counterpart.
- **Effectiveness:** As shown in Table II, applying these conditions improved full-plate accuracy by up to 4.24% for the YOLO26n model. While the character-level accuracy increased marginally ($\approx 1\%$), the impact on the "Plate Accuracy" (where every character must be correct) is significant, proving that syntax validation is essential for real-world deployment.

C. Conclusion

The experimental results indicate that while **YOLO11n-OBB** offers the best detection coverage (mAP_{50}), **YOLO26n-OBB** paired with our conditional recognition pipeline provides the highest end-to-end plate accuracy (70.98%). This combination represents an optimal balance for real-time campus security applications.

VIII. USER INTERFACE IMPLEMENTATION

A. GUI Functionality

A graphical user interface (GUI) was developed to demonstrate and interact with the proposed license plate detection and recognition system. The interface provides several operational modes, including real-time recognition using a webcam, video-based detection, image-based recognition, and exporting processed video results along with CSV data.

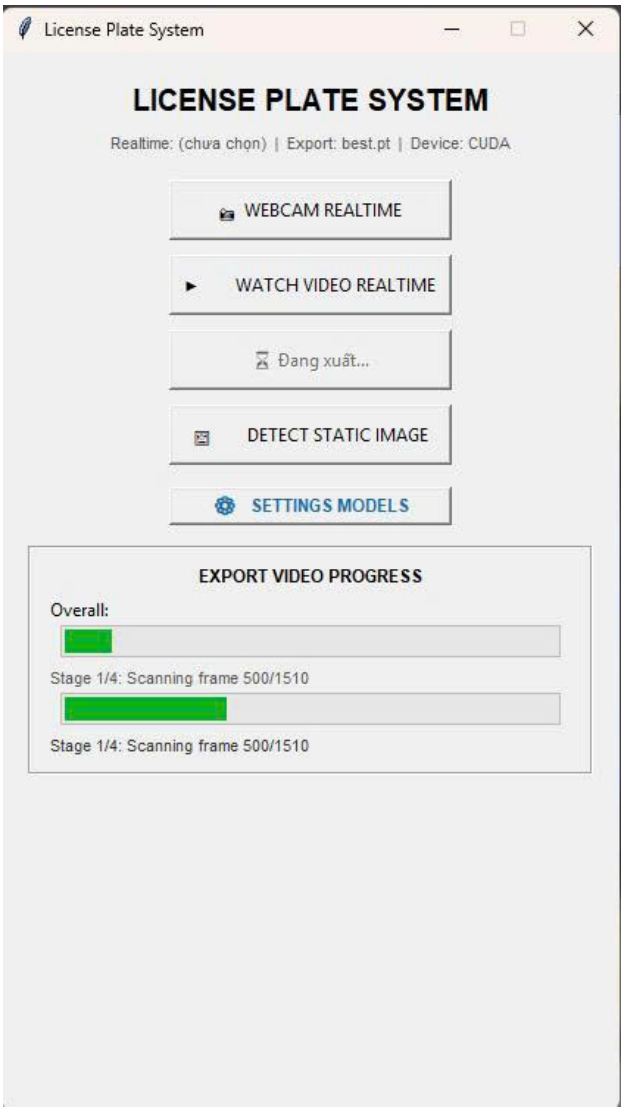

Fig. 3. Main graphical user interface of the proposed license plate recognition system.

The interface enables users to easily select different recognition modes such as webcam realtime detection, video analysis, and static image recognition. The system also allows users to export processed video outputs together with recognized license plate information stored in CSV format.

In addition, a configuration panel is provided to allow users to select and load different trained detection models for realtime preview and batch video processing, as illustrated in. This design makes the system flexible for testing and deployment under different scenarios.

B. Chatbot-Integrated Provincial Mapping

To bridge the gap between raw OCR output and meaningful administrative information, we implemented an Intelligent Query Assistant (Chatbot). This module processes the recognized strings using a Heuristic Mapping Logic based on the Vietnamese vehicle registration standards defined in Circular No. 58/2020/TT-BCA [8].

The chatbot functions as follows:

- **Data Parsing**: Upon recognizing a plate, the chatbot extracts the initial two-digit numeric prefix.
- **Provincial Lookup**: The prefix is matched against a provincial database. For example, prefixes '50' through '59' are mapped to "Ho Chi Minh City," while '29' to '33' correspond to "Hanoi."
- **Natural Language Interaction**: Users can interact with the chatbot to filter history (e.g., "Show all vehicles from Dong Nai detected today").

By incorporating this SBC-enhanced chatbot, the system transforms a simple detection tool into a comprehensive monitoring solution, reducing the cognitive load on security personnel when identifying vehicle origins [9].

IX. Conclusion

This paper presented a robust end-to-end framework for Vietnamese license plate detection and recognition, specifically optimized for high-density and unconstrained environments such as campus parking and urban security. By integrating state-of-the-art YOLO-based Oriented Bounding Box (OBB) architectures with the PaddleOCR engine, we successfully addressed the critical challenge of accurately localizing plates captured from steep, non-orthogonal CCTV angles.Our experimental results demonstrate that the transition to OBB detection provides superior spatial alignment compared to traditional horizontal bounding boxes. Among the evaluated models, **YOLO11n-OBB** achieved a peak mAP_{50} of 0.9923, highlighting its exceptional efficiency in complex plate localization. Furthermore, the introduction of a **Syntax-Based Correction (SBC)** module tailored to Vietnamese vehicle registration standards proved essential for real-world reliability. This post-processing layer yielded a significant improvement in full-plate recognition accuracy—reaching 70.98% for the **YOLO26n-OBB** model—by effectively resolving common alphanumeric ambiguities such as '0' vs. 'D' and '1' vs. 'I'.The implementation of a comprehensive GUI, featuring an Intelligent Query Assistant (Chatbot), further validates the system's practical utility. By automatically mapping recognized numeric prefixes to their respective administrative provinces, the chatbot transforms raw OCR strings into actionable insights for security personnel. While the current nano-architectures provide an optimal balance of speed and precision, future work will focus on further refining the recognition engine for extreme low-light conditions and optimizing the entire pipeline for deployment on low-power edge devices and embedded systems.

References

[1] J. Redmon, S. Divvala, R. Girshick and Ali Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, pp. 779-788, 2016.

[2] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8 and YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics.

[3] S. Chakrabarty, "YOLO26: An Analysis of NMS-Free End to End Framework for Real-Time Object Detection," *arXiv preprint arXiv:2601.04521*, 2026.

[4] Y. Du et al., "PP-OCR: A Practical Ultra Lightweight OCR System," *arXiv preprint arXiv:2009.09941*, 2020.

[5] L. G. Nguyen, D. T. Nguyen and T. V. Nguyen, "Deep Learning-based License Plate Detection and Recognition for Vietnamese Vehicles," *International Conference on Advanced Computing and Applications (ACOMP)*, pp. 112-118, 2021.

[6] X. Yang et al., "On the Arbitrary-Oriented Object Detection: Classification Based Regression and Beyond," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7215-7233, 2022.

[7] R. Laroca et al., "A Robust Real-Time Automatic License Plate Recognition System Based on the YOLO Detector," *International Joint Conference on Neural Networks (IJCNN)*, pp. 1-10, 2018.

[8] K. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[9] M. T. Vo, M. T. Nguyen and H. D. Le, "A Hybrid Approach for Vietnamese License Plate Recognition in Unconstrained Environments," *IEEE Access*, vol. 9, pp. 145021-145035, 2021.

[10] H. Wu et al., "PaddleOCR: Multi-Language OCR Toolset," *Baidu Research*, 2021. [Online]. Available: https://github.com/PaddlePaddle/PaddleOCR.

[11] Z. Ge, S. Liu, F. Wang, Z. Li and J. Sun, "YOLOX: Exceeding YOLO Series in 2021," *arXiv preprint arXiv:2107.08430*, 2021.

[12] T. Dinh, "License Plate Recognition Dataset for Vietnamese Vehicles," GitHub Repository, 2023. [Online]. Available: https://github.com/trungdinh22/License-Plate-Recognition.

[13] S. M. Puranic et al., "Performance Analysis of YOLO Based License Plate Detection," *International Conference on Communication and Signal Processing (ICCSP)*, pp. 0422-0426, 2021.